ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

LAB ASSIGNMENT – 09

NAME – SUFIYA AFREEN

ENROLLMENT NUMBER – 2403A51067

YEAR / SEM – 3/1

DATE – 10-09-26

BRANCH / BATCH – CSE/B-03

Lab/Exp 9: Training a Deeper CNN on a Standard Dataset

Kaggle Dataset Link: https://www.kaggle.com/datasets/ CIFAR-10

1. Load and preprocess the CIFAR-10 dataset.

2. Implement the required Py Torch model.

3. Train using suitable hyperparameters.

4. Evaluate using appropriate metrics.

5. Visualize results (loss/accuracy/features as applicable).

6. Summarize observations and conclusions

CODE –

```
# ============================================================
# LAB / EXP 9
# Training a Deeper CNN on SVHN Dataset
# ============================================================

# 1. IMPORT LIBRARIES
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np

# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Using device:", device)


# ============================================================
# 2. LOAD AND PREPROCESS SVHN DATASET
# ============================================================

# Transform:
# Convert image to Tensor and normalize pixel values
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.4377, 0.4438, 0.4728],
        std=[0.1980, 0.2010, 0.1970]
    )
])

# Training dataset
train_dataset = torchvision.datasets.SVHN(
    root="./data",
    split="train",
    download=True,
    transform=transform
)

# Test dataset
test_dataset = torchvision.datasets.SVHN(
    root="./data",
    split="test",
    download=True,
    transform=transform
)

# Data loaders
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=128,
    shuffle=True,
    num_workers=2
)

test_loader = torch.utils.data.DataLoader(
    test_dataset,
    batch_size=128,
    shuffle=False,
    num_workers=2
)

print("Training samples:", len(train_dataset))
print("Testing samples:", len(test_dataset))


# ============================================================
# 3. VISUALIZE SAMPLE IMAGES
# ============================================================

images, labels = next(iter(train_loader))

# Undo normalization for display
images_display = images.clone()
```

```python
mean = torch.tensor([0.4377, 0.4438, 0.4728]).view(1, 3, 1, 1)
std = torch.tensor([0.1980, 0.2010, 0.1970]).view(1, 3, 1, 1)

images_display = images_display * std + mean
images_display = torch.clamp(images_display, 0, 1)

plt.figure(figsize=(10, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(images_display[i].permute(1, 2, 0))
    plt.title("Label: " + str(labels[i].item()))
    plt.axis("off")

plt.tight_layout()
plt.show()


# ============================================================
# 4. DEFINE DEEPER CNN MODEL
# ============================================================

class DeepCNN(nn.Module):

    def __init__(self):
        super(DeepCNN, self).__init__()

        self.features = nn.Sequential(

            # Block 1
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),

            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),

            nn.MaxPool2d(2),
            nn.Dropout(0.25),

            # Block 2
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),

            nn.MaxPool2d(2),
            nn.Dropout(0.25),

            # Block 3
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            nn.Conv2d(128, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),

            nn.MaxPool2d(2),
            nn.Dropout(0.25)
        )

        self.classifier = nn.Sequential(

            nn.Flatten(),

            nn.Linear(128 * 4 * 4, 256),
            nn.ReLU(),
            nn.Dropout(0.5),

            nn.Linear(256, 10)
        )

    def forward(self, x):
```

```
        x = self.features(x)
        x = self.classifier(x)
        return x


# Create model
model = DeepCNN().to(device)

print(model)


# ============================================================
# 5. LOSS FUNCTION AND OPTIMIZER
# ============================================================

criterion = nn.CrossEntropyLoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=0.001,
    weight_decay=1e-4
)

# Learning-rate scheduler
scheduler = optim.lr_scheduler.StepLR(
    optimizer,
    step_size=5,
    gamma=0.5
)


# ============================================================
# 6. TRAINING
# ============================================================

epochs = 10

train_losses = []
train_accuracies = []

test_losses = []
test_accuracies = []


for epoch in range(epochs):

    # -------------------------
    # Training
    # -------------------------
    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:

        images = images.to(device)
        labels = labels.to(device)

        # Forward pass
        outputs = model(images)

        loss = criterion(outputs, labels)

        # Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    train_loss = running_loss / len(train_loader)
```

```
    train_accuracy = 100 * correct / total

    train_losses.append(train_loss)
    train_accuracies.append(train_accuracy)


    # -------------------------
    # Testing
    # -------------------------
    model.eval()

    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():

        for images, labels in test_loader:

            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)

            loss = criterion(outputs, labels)

            running_loss += loss.item()

            _, predicted = torch.max(outputs.data, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    test_loss = running_loss / len(test_loader)
    test_accuracy = 100 * correct / total

    test_losses.append(test_loss)
    test_accuracies.append(test_accuracy)

    scheduler.step()

    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"Train Loss: {train_loss:.4f} "
        f"Train Acc: {train_accuracy:.2f}% "
        f"Test Loss: {test_loss:.4f} "
        f"Test Acc: {test_accuracy:.2f}%"
    )


# ============================================================
# 7. FINAL TEST ACCURACY
# ============================================================

print("\nFinal Test Accuracy: {:.2f}%".format(test_accuracies[-1]))


# ============================================================
# 8. VISUALIZE LOSS
# ============================================================

plt.figure(figsize=(8, 5))

plt.plot(train_losses, label="Training Loss")
plt.plot(test_losses, label="Testing Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Testing Loss")

plt.legend()
plt.grid()
plt.show()


# ============================================================
# 9. VISUALIZE ACCURACY
# ============================================================
```

```python
plt.figure(figsize=(8, 5))

plt.plot(train_accuracies, label="Training Accuracy")
plt.plot(test_accuracies, label="Testing Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.title("Training and Testing Accuracy")

plt.legend()
plt.grid()
plt.show()


# ============================================================
# 10. DISPLAY PREDICTIONS
# ============================================================

model.eval()

images, labels = next(iter(test_loader))

images_gpu = images.to(device)

with torch.no_grad():
    outputs = model(images_gpu)
    _, predictions = torch.max(outputs, 1)

# Convert images back for display
images_display = images.clone()
images_display = images_display * std + mean
images_display = torch.clamp(images_display, 0, 1)

plt.figure(figsize=(12, 6))

for i in range(12):

    plt.subplot(3, 4, i + 1)

    plt.imshow(images_display[i].permute(1, 2, 0))

    plt.title(
        f"Actual: {labels[i].item()} | "
        f"Pred: {predictions[i].item()}"
    )

    plt.axis("off")

plt.tight_layout()
plt.show()
```

OUTPUT –

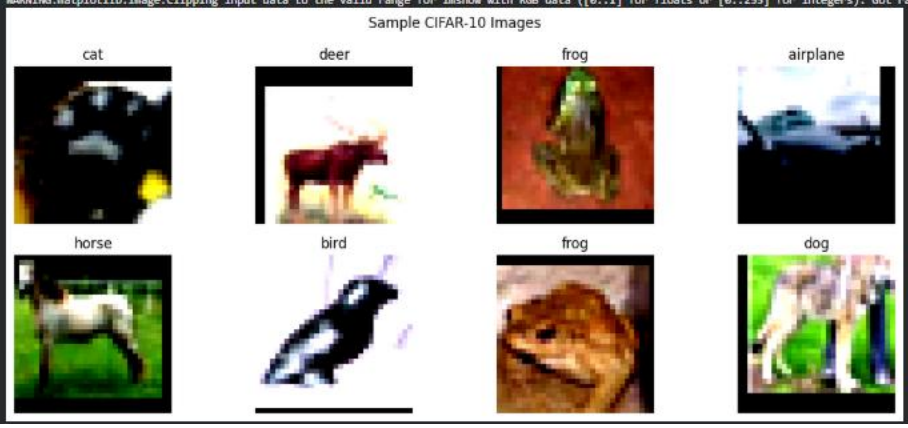
```
Using device: cpu
100%|██████████| 182M/182M [00:18<00:00, 9.87MB/s]
100%|██████████| 64.3M/64.3M [00:07<00:00, 8.34MB/s]
Training samples: 73257
Testing samples: 26032
```




```
DeepCNN(
  (features): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU()
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (7): Dropout(p=0.25, inplace=False)
    (8): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU()
    (11): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): ReLU()
    (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (15): Dropout(p=0.25, inplace=False)
    (16): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (17): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (18): ReLU()
    (19): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (20): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (21): ReLU()
    (22): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (23): Dropout(p=0.25, inplace=False)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=2048, out_features=256, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=256, out_features=10, bias=True)
  )
)
```

```
Using device: cpu
100%|██████████| 170M/170M [18:38<00:00, 152kB/s]

Training images: 50000
Testing images: 10000
Number of classes: 10
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.71453285..1.8183348].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.71453285..1.8768656].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.71453285..1.1198891].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.71453285..1.4573944].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.71453285..1.6705607].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.71453285..1.8573554].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.71453285..1.1754966].
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.71453285..1.650427].
```




```
================ MODEL ================
DeepCNN(
  (conv1): Sequential(
    (0): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Conv2d(32, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU()
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (conv2): Sequential(
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU()
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (conv3): Sequential(
    (0): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (4): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (5): ReLU()
    (6): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (fc): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=2048, out_features=256, bias=True)
    (2): ReLU()
    (3): Dropout(p=0.5, inplace=False)
    (4): Linear(in_features=256, out_features=10, bias=True)
  )
)

================ TRAINING ================

Epoch [1/10]  Loss: 1.5314  Accuracy: 43.74%
```